Guardianes del estero

Simulador funcional

Asignatura: Programación Funcional

Actividad integradora: Unidades 1 y 2

1. Introducción

Guardianes del Estero es una aplicación de escritorio desarrollada en Python que simula la organización de brigadas comunitarias para atender distintas necesidades de un manglar costero. El usuario puede seleccionar una zona y ejecutar acciones de limpieza, rescate de fauna o reforestación. Cada acción actualiza el estado de la simulación y registra lo ocurrido.

El proyecto integra una interfaz gráfica con Tkinter y ttkbootstrap, además de conceptos de programación funcional e imperativa. La lógica se mantiene separada de los elementos visuales para facilitar su comprensión.

2. Objetivos

Objetivo general

Construir una aplicación gráfica que permita explorar conceptos de programación funcional mediante una simulación interactiva de conservación ambiental.

Objetivos específicos

- Representar brigadas, zonas y órdenes mediante estructuras de datos claras.
- Aplicar funciones puras, funciones de orden superior, lambda, cierres y transformaciones inmutables.
- Usar match/case para decidir qué tipo de orden debe ejecutarse.
- Comparar una solución funcional con un procedimiento imperativo.
- Separar la lógica del dominio, el estado de la simulación y la interfaz gráfica.

3. Descripción de la aplicación

La aplicación permite seleccionar una zona y una brigada, y ejecutar una de estas acciones:

- Limpiar: reduce los residuos acumulados en una zona.
- Rescatar: disminuye el número de animales que requieren atención.
- Reforestar: aumenta la cantidad de plántulas o árboles registrados.

La simulación también considera la energía disponible de las brigadas y conserva una bitácora de las acciones realizadas.

4. Diseño y arquitectura

El proyecto se divide en tres responsabilidades:

- Modelo de datos: representa brigadas, zonas, órdenes y estados.
- Lógica funcional: realiza cálculos, filtros, resúmenes y procesamiento de órdenes sin depender de los widgets.
- Interfaz gráfica: recibe las elecciones del usuario, llama a la lógica y actualiza la pantalla.

5. Aplicación de la Unidad 1

Funciones puras

Las funciones de cálculo reciben datos y producen resultados sin modificar directamente la interfaz ni realizar operaciones externas. Por ejemplo, el cálculo del impacto y los resúmenes pueden evaluarse con los mismos datos para obtener el mismo resultado.

Funciones de orden superior y lambda

Se utilizan map, filter y reduce para transformar colecciones, seleccionar elementos y acumular valores. Lambda se reserva para operaciones pequeñas y fáciles de entender.

Cierre o closure

La función crear_medidor_impacto recibe un multiplicador y devuelve otra función que conserva ese valor. La función interna utiliza el multiplicador para calcular el impacto.

Comparación funcional e imperativa

El proyecto incluye un cálculo imperativo paso a paso para compararlo con una alternativa funcional. La versión imperativa utiliza instrucciones secuenciales y variables intermedias; la funcional expresa el cálculo mediante funciones y transformaciones de datos.

6. Aplicación de la Unidad 2

Modelado de datos e inmutabilidad

Las entidades se representan mediante dataclass. Se usa frozen=True para impedir la modificación directa de sus campos. Cuando cambia el estado, se crea una nueva versión del objeto.

match/case

El procesamiento de órdenes utiliza match/case para distinguir entre limpieza, rescate y reforestación. Cada caso aplica las reglas correspondientes.

Funciones puras y efectos secundarios

Los cálculos se mantienen separados de las operaciones que actualizan la pantalla o reciben eventos. La interfaz realiza los efectos secundarios necesarios, mientras que la lógica se puede probar por separado.

7. Uso de map, filter y reduce

map: genera resúmenes de las zonas al transformar cada elemento en una representación útil.

filter: selecciona las zonas que cumplen una condición de prioridad.

reduce: combina valores de varias zonas para obtener un total de contaminación.

8. ¿Por qué no usar lambda para todo?

Lambda es útil para operaciones breves, pero las reglas con varias decisiones o validaciones son más legibles y fáciles de probar con funciones definidas mediante def. Por eso el proyecto reserva lambda para transformaciones cortas y usa funciones con nombre para la lógica principal.

9. Pruebas

- La ventana abre correctamente y los controles se muestran completos.
- Se puede elegir una zona y una brigada.
- La limpieza reduce los residuos de la zona seleccionada.
- El rescate reduce la cantidad de fauna que requiere atención.
- La reforestación aumenta el registro de plántulas o árboles.
- Las órdenes inválidas o sin energía suficiente muestran un mensaje comprensible.
- La bitácora y los indicadores se actualizan después de una acción.

Captura de evidencia: [Inserta aquí una captura real de la aplicación funcionando, si la actividad lo solicita].

10. Uso de inteligencia artificial

Se utilizó una herramienta de inteligencia artificial (codex) como apoyo para organizar la estructura inicial, proponer ejemplos de conceptos funcionales y preparar la documentación. El código y los documentos deben revisarse y probarse antes de entregar.

- La simulación relaciona conceptos funcionales con un problema ambiental concreto.
- Los datos inmutables y las transformaciones explícitas facilitan comprender los cambios de estado.
- La comparación funcional e imperativa muestra que una tarea puede expresarse de distintas maneras.

Enlaces de entrega

Repositorio de GitHub: https://github.com/IzaguirreVictor/guardianes-del-estero

Enlace video: https://www.youtube.com/watch?v=dQw4w9W